



UNLOX

Monitoring ML Models in Production

Detecting drift, preserving performance, and keeping your models reliable long after deployment.



UNLOCK

CHAPTER 29.1

The Need for Monitoring

Deploying a model is not the finish line — it's the starting line. Once live, a model operates in a dynamic environment where the data it encounters will inevitably shift over time. User behavior changes, upstream data pipelines evolve, and real-world conditions fluctuate seasonally or unexpectedly. These shifts cause **model drift**: a gradual or sudden degradation in prediction quality that can go unnoticed without active surveillance.

Continuous monitoring ensures that you catch performance decay before it impacts business outcomes, customer trust, or downstream systems that depend on your model's output.

UNLOCK

UNDERSTANDING DRIFT

What Exactly Is Model Drift?

Data Drift

The statistical distribution of input features changes relative to the training data. For example, a fraud model trained on pre-pandemic transaction patterns may see entirely new spending behaviors post-pandemic.

Concept Drift

The relationship between features and the target variable itself changes. What used to predict churn no longer does because customer expectations have shifted in the market.

Prediction Drift

The distribution of model outputs shifts over time, even if inputs appear stable. This can signal hidden degradation in the decision boundary or feature importance.

UNLOCK

WHY IT MATTERS

The Cost of Ignoring Drift

Silent Failures

Unlike traditional software bugs that crash loudly, model drift fails silently. Predictions still return — they're just wrong. Without monitoring, teams discover degradation only after significant business damage has occurred.

Real-World Consequences

- **Revenue loss** from inaccurate recommendations or pricing
- **Compliance violations** when fairness metrics decay undetected
- **Eroded trust** among stakeholders who rely on model outputs
- **Cascading failures** in downstream systems consuming stale predictions
- **Wasted compute** serving a model that no longer delivers value



UNLOCK

CHAPTER 29.2

Tools for Drift Detection

A robust monitoring stack combines specialized drift detection libraries with real-time visualization platforms. Together, they give MLOps teams both the analytical depth to diagnose issues and the operational dashboards to act on them quickly. Let's explore two essential tools in this space.

UNLOCK

TOOL DEEP-DIVE

Evidently AI



Interactive Drift Reports

Evidently generates rich HTML reports that visualize feature-level distribution shifts. You can compare a reference dataset against current production data to pinpoint exactly which features have drifted and by how much.



Data & Model Drift Analysis

The framework tracks both **data drift** (input distribution changes) and **model drift** (performance metric degradation). Statistical tests like Kolmogorov-Smirnov and Population Stability Index run automatically across all features.



Pipeline Integration

Evidently plugs directly into CI/CD and orchestration tools like Airflow or Prefect. Run drift checks as automated pipeline steps that gate retraining triggers or alert the on-call engineer.

UNLOCK

TOOL DEEP-DIVE

Grafana



Real-Time Dashboards

Grafana excels at building live dashboards that track key production metrics — accuracy, precision, recall, latency, and throughput — all updating in real time. Teams get a single pane of glass for model health.



Alerting & Thresholds

Configure custom alert rules that fire when metrics cross predefined thresholds. Notifications route to Slack, PagerDuty, or email so the right people respond immediately when model performance degrades.



Flexible Data Sources

Grafana connects to Prometheus, InfluxDB, PostgreSQL, and dozens of other backends. This means you can unify model metrics alongside infrastructure and application metrics in a single observability layer.

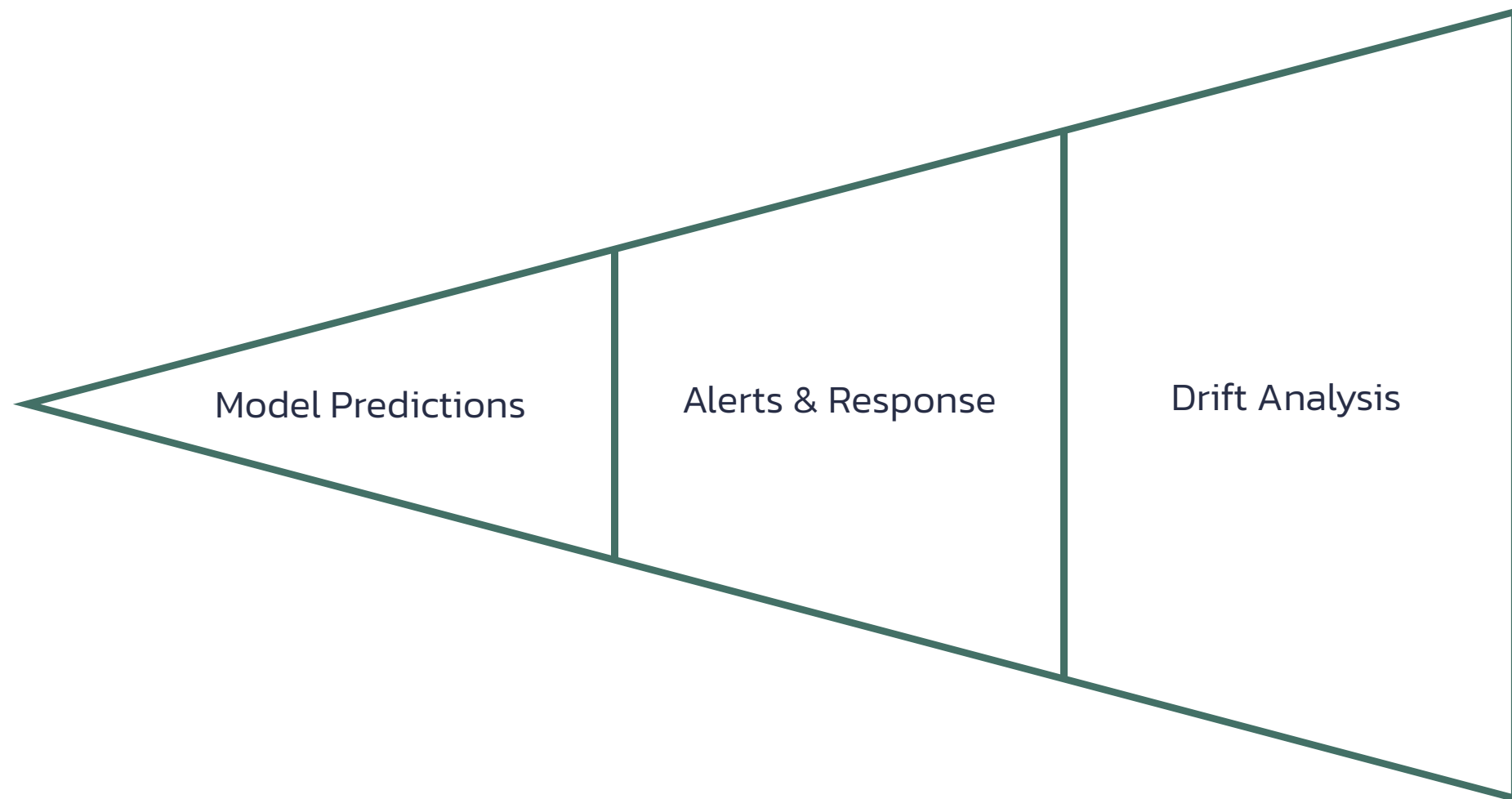
Evidently AI vs. Grafana: When to Use Each

These tools are complementary, not competitive. Understanding their strengths helps you architect a monitoring stack that covers both analytical depth and operational visibility.

Dimension	Evidently AI	Grafana
Primary Focus	Statistical drift detection & ML-specific analysis	Real-time metric visualization & alerting
Output	Interactive HTML/JSON reports	Live dashboards & configurable alerts
Best For	Deep-dive diagnostics on feature distributions	Continuous health monitoring at a glance
Integration	ML pipelines, notebooks, CI/CD	Prometheus, databases, cloud services
Licensing	Open-source (Apache 2.0)	Open-source (AGPL v3) + cloud option

Putting It All Together

A production-grade monitoring architecture combines both tools into a seamless workflow. Here's how data flows from your model to actionable alerts.



This architecture ensures that drift is detected statistically by Evidently, surfaced visually in Grafana, and acted upon by the team — closing the feedback loop between deployment and retraining.

Monitoring Best Practices

1

Define Baselines Early

Capture reference distributions and performance metrics at deployment time. Without a clear baseline, you cannot measure drift meaningfully.

2

Monitor Inputs & Outputs

Track both feature distributions and prediction distributions. Input drift can precede output degradation, giving you an early warning signal.

3

Automate Retraining Triggers

Link drift alerts to automated retraining pipelines. When statistical tests exceed thresholds, kick off a new training run with fresh data automatically.

4

Log Everything

Store predictions, features, and ground truth labels. This audit trail is essential for debugging, compliance, and continuous improvement of your monitoring thresholds.

UNLOCK

KEY TAKEAWAYS

Key Takeaways & Next Steps

Drift Is Inevitable

Every production model will experience drift. The question is not *if* but *when* — so build monitoring from day one.

Use the Right Tool

Pair **Evidently AI** for deep statistical analysis with **Grafana** for real-time dashboards and alerting.

Close the Loop

Connect monitoring signals to automated retraining pipelines so your models self-heal before users feel the impact.

